

Claude Code 勉強会

基礎から自分専用AIチームまで

普段AIに触っている／Claude Codeで動画編集している
でも基礎がフワッとしている人 のための1時間

今日のゴール & タイムライン (60分)

ゴール

-  Claude Code が「ただのチャットAI」と何が違うか分かる
-  `.claude/` フォルダの中身を **迷わず編集** できる
-  自分の業務を「毎回説明しないでも動く形」にできる
-  専門家チームを作って、安く・速く・安全に動かせる

時間	内容
0-15分	Claude Codeとは／3つの登場人物
15-25分	<code>CLAUDE.md</code> と <code>settings.json</code>
25-40分	スキルを作る・育てる
40-50分	エージェント・HITL
50-55分	コスト管理
55-60分	ハンズオン課題と振り返り

第1部

Claude Code とは何か

普通のChatGPT・Claudeとの違い

普通のチャットAI	Claude Code
文章で答えるだけ	ファイルを読み・書き・実行
あなたがコピペで橋渡し	PC上で完結
毎回ゼロから説明	プロジェクトのルールを覚える
1人で何でもやらされる	専門家チームに分業

動画編集者にとっての効きどころ

- Premiere Pro の XML を読んで、無音カット・ジェットカットを自動化
- 文字起こし → SRT 字幕生成 → 改行整形までを自動チェーン
- 動画から note 記事 / X投稿文 / YouTube 概要欄まで自動下書き

インストール・起動・基本コマンド

```
# 標準 : npm でグローバルインストール
npm install -g @anthropic-ai/claude-code

# native install (提供状況は公式で要確認)
curl -fsSL https://claude.ai/install.sh | bash

# プロジェクトのフォルダに移動して起動
cd ~/your-project && claude
```

⚠ 最新の標準手順は[公式ドキュメント](#) で必ず確認

コマンド	何をする
/init	CLAUDE.md のたたき台を自動生成 ★最初の一手
/cost	今のセッションのコスト確認
/clear	会話履歴リセット
/memory	プロジェクトの記憶を編集
/help	コマンド一覧

第2部

最初に押さえる「3つの登場人物」

3つの登場人物 — 1行で違いを言うと

登場人物	役割	ざっくり例え
CLAUDE.md	プロジェクトのルール書	会社の就業規則
skills/	自動で発動する手順書	バイト用マニュアル
agents/	専門家として呼ばれる別AI	外注の専門家

- CLAUDE.md = 「うちのチームではこう動いてね」 → 常駐ルール
- skills/ = 「議事録を頼まれたらこの手順で動いてね」 → 発動型マニュアル
- agents/ = 「調査だけ別のAIに任せたい」 → 分業先

.claude/ フォルダ全体像

your-project/	
├── CLAUDE.md	← セッション開始時に必ず読まれる
├── CLAUDE.local.md	← 個人設定（チーム共有しない）
├── .claude/	
│ ├── settings.json	← ツール権限のallow/deny
│ ├── commands/	← /xxx で明示的に呼ぶコマンド
│ ├── skills/	← 会話から自律発動するワークフロー
│ └── agents/	← 専門分業のサブエージェント

💡 CLAUDE.md は「常時ON」、skills/agents は「呼ばれたら動く」

第3部

CLAUDE.md と settings.json

CLAUDE.md = AIの「就業規則」

セッション開始時に **必ず** 読まれ、AIの頭の中にずっと居続ける
だから **短く・具体的に** が鉄則

✗ ダメな例（抽象的）：

丁寧に質問に答えてください。コードは綺麗にしてください。

○ 良い例（トリガー＋アクション）：

- レビューを依頼されたら、重大リスクから先に列挙すること
- 外部送信の前に、必ず実行前確認を行うこと
- タスク更新時は原本ファイルのみ更新すること

削除テストと200行ルール

削除テスト：「この行を消したら何か困る？」
答えが「特に困らない」なら、その行は不要

200行以内に収める

CLAUDE.md は毎セッション読まれる

長い = 毎回トークンを払う = 出力もブレる

→ 詳細ルールは `skills/` 配下に逃がす（後述の Progressive disclosure）

settings.json — 安全設計の入り口

```
{
  "permissions": {
    "allow": ["Bash(git status)", "Bash(npm test)"],
    "deny":  ["Read(../.env)", "Bash(rm -rf *)"]
  }
}
```

設定	意味
allow	確認なしで実行
deny	実行を禁止（最優先）
未指定	実行前に確認

💡 最初に書くべき1行: `"deny": ["Read(../.env)"]`

第4部

スキルを作る・育てる

スキルとは？最小ファイル例

特定の依頼が来たら、AIが自動で発動する手順書

```
.claude/skills/字幕生成/  
└─ SKILL.md
```

```
---  
name: 字幕生成  
description: WAV音声と動画から、Premiere Pro用のSRT字幕を生成する  
---
```

手順

1. 音声を Whisper で文字起こし
2. 句読点で改行を整える
3. SRT形式で `output/` に書き出す

`description` を頼りに、Claude が自動で発動する

★ Gotchas — 失敗パターンを蓄積する

Claude Code 利用者・開発者の記事で「最も価値が高い」とよく挙げられるのが Gotchas

(※ 公式必須項目ではなく、コミュニティで広く支持される実践知)

スキル実行 → AIがミス → 「なぜミスしたか」を1行追記 → 次回から出なくなる

Gotchas(字幕生成スキルの例)

- 5000文字超だと改行判断が荒くなる → 1000文字ずつ分割
- 「えーと」「あのー」のフィラーを残す → 削除指示を明示
- SRTのタイムコードがズレる → ffprobe で実尺を再計算

最初から完璧に書く必要なし。1行ずつ追記 で使うほど賢くなる

Progressive disclosure — フォルダ分割設計

SKILL.md に全部書くと 重い・高い・精度が落ちる

```
.claude/skills/字幕生成/
├── SKILL.md           ← 概要・トリガー・基本手順だけ（毎回読まれる）
├── references/
│   ├── format_rules.md ← 詳細フォーマット（必要時のみ参照）
│   └── examples.md     ← 過去の良い例
├── assets/template.srt ← テンプレート
└── data/log.json       ← 実行ログ（永続データ）
```

SKILL.md には一言だけ：

詳細フォーマットは references/format_rules.md を参照

データ永続化 & スキル連携チェーン

永続化： `data/log.json` に追記指示を書くだけ

- 毎回の結果を `data/log.json` に追記
- 次回実行時に前回との差分を検出

→ 毎朝のタスク通知、週次レポート、先週比の自動計算

連携： `description` に「〇〇の後に実行」と書くだけ

X投稿文スキル

`description`: 動画完成後、SNS展開を依頼されたときに実行する

→ 動画完成 → X投稿文 → note記事 → YouTube概要欄

第5部

エージェント — AIチームを作る

エージェント vs 万能AI vs スキル

エージェント = ゴールを与えると、自律的に計画→実行→確認→修正を繰り返す別AI

	万能AI 1人	専門家チーム
コンテキスト	情報が混ざって汚染	各自独立・クリーン
精度	役割が多いと落ちる	専門特化で高精度
ツール権限	全部	必要な分だけ

スキル vs エージェント

- スキル：メインClaudeが自分で実行、メインとコンテキスト共有
- エージェント：別のAIに委譲、コンテキスト完全独立、モデル使い分け可

エージェントファイルの書き方

```
---  
name: researcher  
description: Web検索・情報収集が必要なときに自動委譲される  
tools: WebSearch, WebFetch, Read  
model: <軽量モデル例>  
---
```

役割

情報収集に特化。要約・判断はせず、収集した情報をそのまま返す。

- `description` の条件で 自動的に呼ばれる
- あなたは常にメインのClaudeと話すだけ

委譲パターン — 逐次 / 並列

A. 逐次委譲（直列チェーン） — 工程に順番がある業務向け

```
ユーザー → メイン → researcher → writer → reviewer → 完成物
```

例：議事録、提案書、動画→記事化

B. Fork-Join 並列 — 分解できるタスク向け

```
メイン: 3体同時起動 (Fork) → 全員完了後にメインが統合 (Join)  
researcher_A / researcher_B / researcher_C
```

例：複数候補比較、複数チャンネル分析、複数記事一括校正

タスクを小さく分解できるほど並列の効果が増す

第6部

HITL — 人間が止めるべき場所

HITLとは — 黄金律

AIが自動で動く流れの中に、人間が確認するタイミングを意図的に組み込む
全自動はリスク大、全手動は意味なし、中間を設計 する

黄金律

「取り消せないか？ 自分以外に影響が及ぶか？」
→ YES なら確認を入れる

	取り消せる	取り消せない
影響小	自動でOK	確認推奨
影響大	確認推奨	必ず確認 (MUST)

どこに書くか — 操作に最も近いファイルへ

ファイル	何を書くか
CLAUDE.md	コア原則 1行のみ
各スキル	そのスキル固有のHITL
各エージェント	そのエージェント固有のHITL

```
# CLAUDE.md
- 取り消せない操作は必ず確認してから実行すること
```

```
# agents/mailler.md
- 送信前に必ず内容を表示して「送信してよいですか？」と確認
- 絶対に確認なしで実行しないこと
```

シーン別 HITL の入れ方

操作	HITLの入れ方
Slack/メール送信	内容を表示して承認
ドキュメント/Wiki更新	差分を表示して確認
ファイル削除	削除対象リストを提示
5段階以上の長いタスク	3段階目で中間確認
動画書き出し（上書き）	書き出し先パスを必ず表示

「待て」「検証しろ」の2ルールでやり直し率が大幅減（コミュニティ報告）

第7部

コストとモデル使い分け

モデル選び — 相対的な使い分け

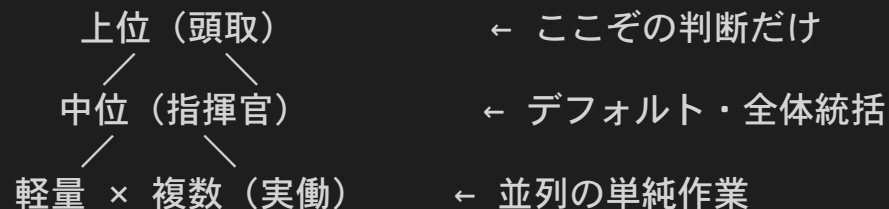
Claude には用途別に **複数のモデル** がある（具体名・価格は[公式 Pricing](#) を参照）

階層	役割の目安	向いている業務
上位（最高精度）	重要な意思決定・複雑推論	設計・影響大な文章
中位（汎用）	自律実行・執筆	議事録・記事・コード
軽量（高速・安価）	大量・単純タスク	情報収集・データ整形・Sub-agent

使い分けの目安：軽い調査や整形は安価なモデル、重要判断だけ上位モデル

コスト削減3つのワザ & 設計ピラミッド

1. プロンプトキャッシュ — `CLAUDE.md` 等の不変部分をキャッシュ
2. バッチ処理 — リアルタイム不要なタスクをまとめる
3. 軽量モデルへの委譲 — Sub-agent の単純作業を安いモデルに



- `/cost` で確認 → 軽量モデル移行候補を1つ見つけたら勝ち
- 詳細・最新価格は[公式 Pricing](#) で要確認

全体設計の最終形

今回の設計を1つのフォルダに落とし込む

```
your-project/
├── CLAUDE.md          ← コア原則のみ（200行以内）
├── .claude/
│   ├── settings.json ← deny: rm -rf *, .env 読み取り
│   ├── commands/     ← /xxx で明示的に呼ぶ
│   ├── skills/
│   │   └── 字幕生成/
│   │       ├── SKILL.md      ← 概要+Gotchas+HITL
│   │       └── references/
│   │           └── data/log.json ← 永続化
│   └── agents/
│       ├── researcher.md ← HITL不要（読み取りのみ）
│       ├── writer.md     ← HITL不要（ドラフト作成）
│       └── mailer.md      ← HITL必須！（送信＝取り消せない）
```

ハンズオン課題

最低1つやって帰ってください

Lv.1 / Lv.2 / Lv.3

Lv.1 (10分) 今すぐできる

- [] `/init` で `CLAUDE.md` のたたき台を生成
- [] `settings.json` に `"deny": ["Read(../.env)"]`
- [] `/cost` で今のセッションコスト確認

Lv.2 (30分) 自分のスキルを作る

- [] 「毎回説明していること」を1つ思い出して `skills/` に書く
- [] そのスキルに **Gotchas** を3行 書く

Lv.3 (60分) チーム化

- [] スキルを Progressive disclosure に整える
- [] `agents/researcher.md` を1つ作る
- [] HTTP ルールを 送信系エージェント に書く

今日から使うための次アクション

今日のうちに (5分でOK)

- `/init` を打って `CLAUDE.md` のたたき台を作る
- `settings.json` に `"deny": ["Read(./env)"]` を1行入れる
- `/cost` で今のセッションコストを眺めてみる

今週のうちに (30分)

- 自分の業務で「毎回説明していること」を1つ `skills/` 化
- そのスキルに **Gotchas** を3行 だけ書く
- 1度動かしてみて、ミスを `Gotchas` に1行追記

育て方の合言葉

完璧を目指さない、1行ずつ追記する

使うほど賢くなるAIは、こうして手元に育つ

振り返り

-  Claude Code = ファイル操作できる別格のAI
-  CLAUDE.md / skills / agents の3層で設計
-  Gotchas で「使うほど賢く」育てる
-  HITL で「取り消せない操作」を守る
-  軽量／中位／上位モデルの3層ピラミッドでコスト管理

おつかれさまでした 🎉

質問・つまづき はDiscordへ
最新版資料 はGitHubで更新中



解説MD : `README.md`



このスライド : `slides.md`